

Data Representation

- ① Decimal no. system (0-9) Base 10
- ② Binary no. system (0, 1) Base 2
- ③ Octal no. system (0-7) Base 8
- ④ Hexadecimal no. system (0-15) Base 16

1) Decimal to Binary
 $(97)_{10} \rightarrow (\quad)_2$

		Remainder
2	97	1
2	48	0
2	24	0
2	12	0
2	6	0
2	3	1
	1	

$(1100001)_2$



How to check this is correct?

Binary to Decimal

$$1 \times 2^6 + 0 \times 2^5 + 0 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$

$$\Rightarrow 64 + 32 + 0 + 0 + 0 + 0 + 1$$

$$= (97)_{10}$$

2) Decimal to Octal

$(97)_{10} \rightarrow (\quad)_8$

		Remainder
8	97	1
8	12	4
	1	

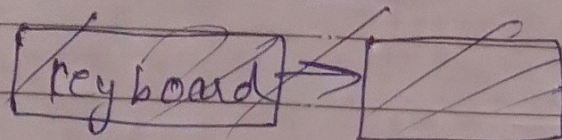
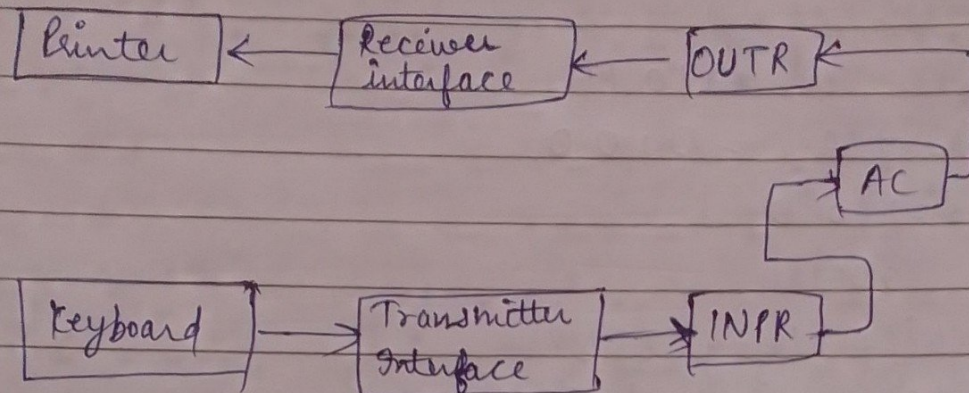
$(141)_8$

3) Decimal to Hexadecimal

$(97)_{10} \rightarrow (\quad)_{16}$



#

Input and OutputF₀OF₀I

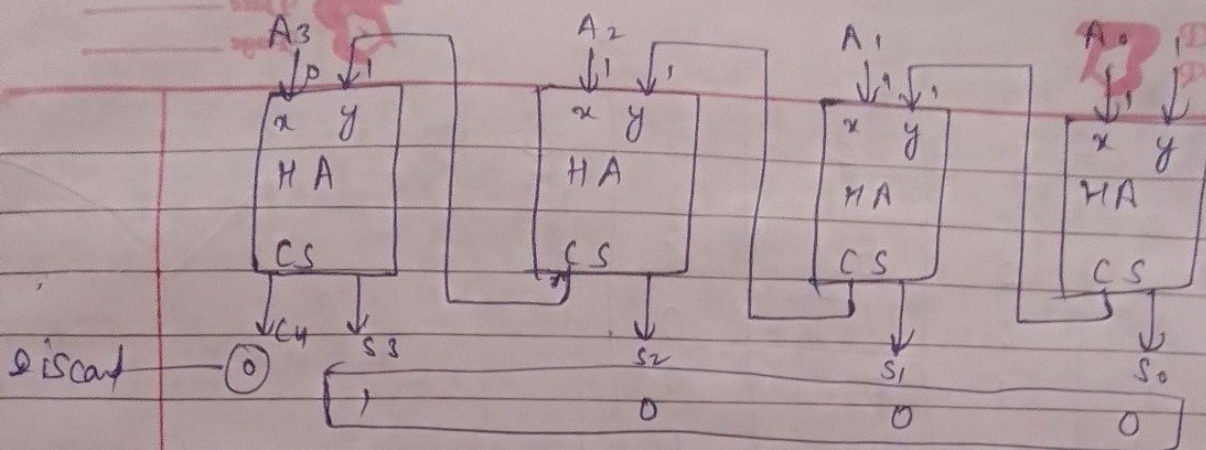
keyboard works on serial input.

Two flip flop :- 1) F₀O ← Output Flag
2) F₀I ← Input Flag

- when F₀I = 0, (key press kake) any input can be entered through keyboard

- F₀I value = 1 when we are done with entering data (New data can't be entered until current is stored)

initialization



Binary Adder and Subtractor:-

$$A = 1011, B = 0101$$

If $M = 0$ then perform Addition

If $M = 1$ then perform Subtraction

Case I: $M = 0$

$$A = 1011 = (11)_2 = 1 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$

$$B = 0101 = (5)_2 = 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$10000 = (16)_2 = 1 \times 2^4 + 0 \times 2^3 + 0 \times 2^2 + 0 \times 2^1 + 0 \times 2^0$$

$$= 1 \times 2^3 + 0 \times 2^2 + 1 \times 2^1 + 1 \times 2^0$$

$$= 8 + 2 + 1$$

$$= 11$$

$$0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0$$

$$= 4 + 1$$

$$= 5$$

Case II: $M = 1$

$$A = 1011$$

$$B = 0101$$

$$A = 1011$$

$$B = 0101 \rightarrow$$

$$1010 \rightarrow 1's \text{ complement}$$

$$+ 1$$

$$1011 \rightarrow 2's \text{ complement}$$

$$A = 1011$$

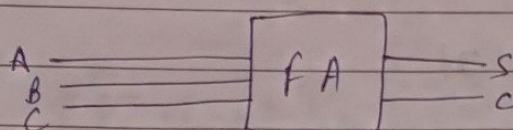
$$1011 - 2's \text{ complement}$$

Discard

$$0110$$

$$0110 \rightarrow$$

A	B	C	S	C
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1



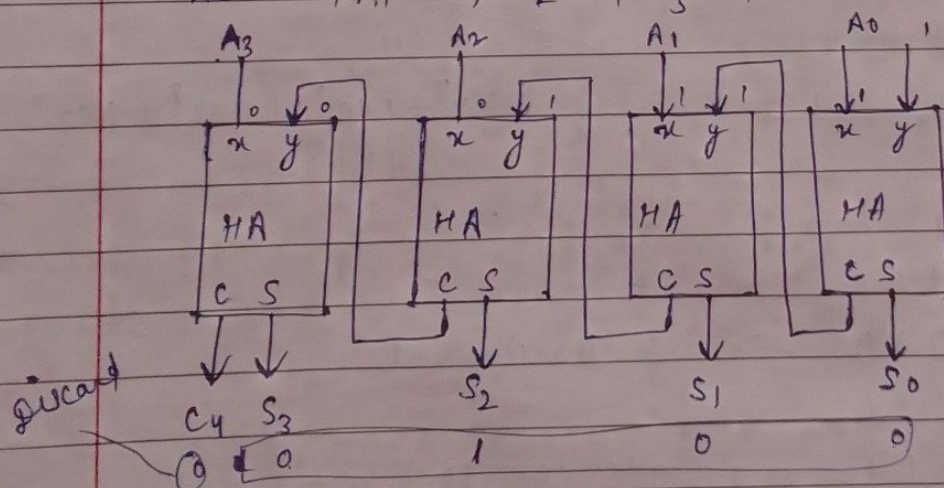
(2 bit add - Half adder)

Binary Incrementer

$$\text{original} \leftarrow A = \begin{array}{r} 0011 \\ + 1 \\ \hline \end{array}$$

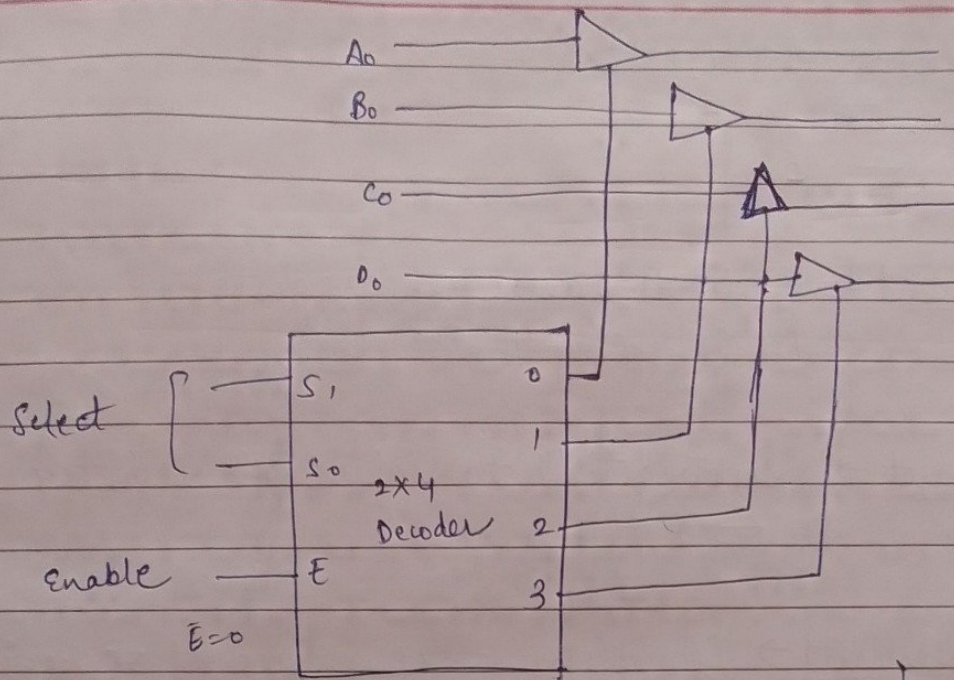
$$\text{Incrementer} \leftarrow \begin{array}{r} 0100 \end{array}$$

$$A_0 = 0, A_1 = 0, A_2 = 1, A_3 = 1$$



$$\begin{array}{r} 0011 \\ + 1 \\ \hline 0100 \end{array}$$

$$A = \begin{array}{r} 0111 \\ + 1 \\ \hline 1000 \end{array}$$



S_0	S_1
0	0
0	1
1	0
1	1

Binary Adder

When 2 bits are added then half adder is used. When 2 bits ~~along~~ with carry are added then full adder is used.

(3 bits as input)
Half Adder

A	B	S	C
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

$2^2 = 4$ combination

In full adder $= 2^3 = 8$ combination

Full Adder

A	B	C	S	C
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0

$$R_3 \leftarrow R_1 - R_2$$

$$R_3 \leftarrow 10 - 5$$

$$R_3 \leftarrow 5$$

3.) Increment $\rightarrow$ Add 1 to value of register
 $R_1 \leftarrow R_1 + 1$

4.) Decrement $\rightarrow$ subtract 1 to value of register
 $R_1 \leftarrow R_1 - 1$

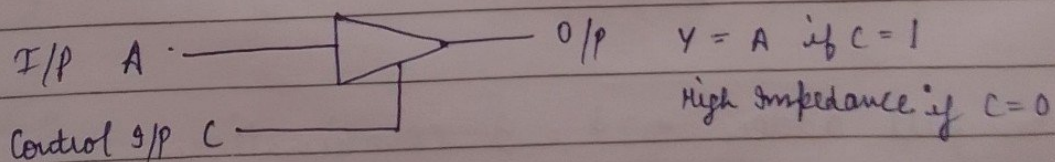
5.) 1's complement $\rightarrow$ Complement of value of register
 $R_1 \leftarrow \bar{R}_1$
 (i.e. $0 \rightarrow 1$
 $1 \rightarrow 0$)

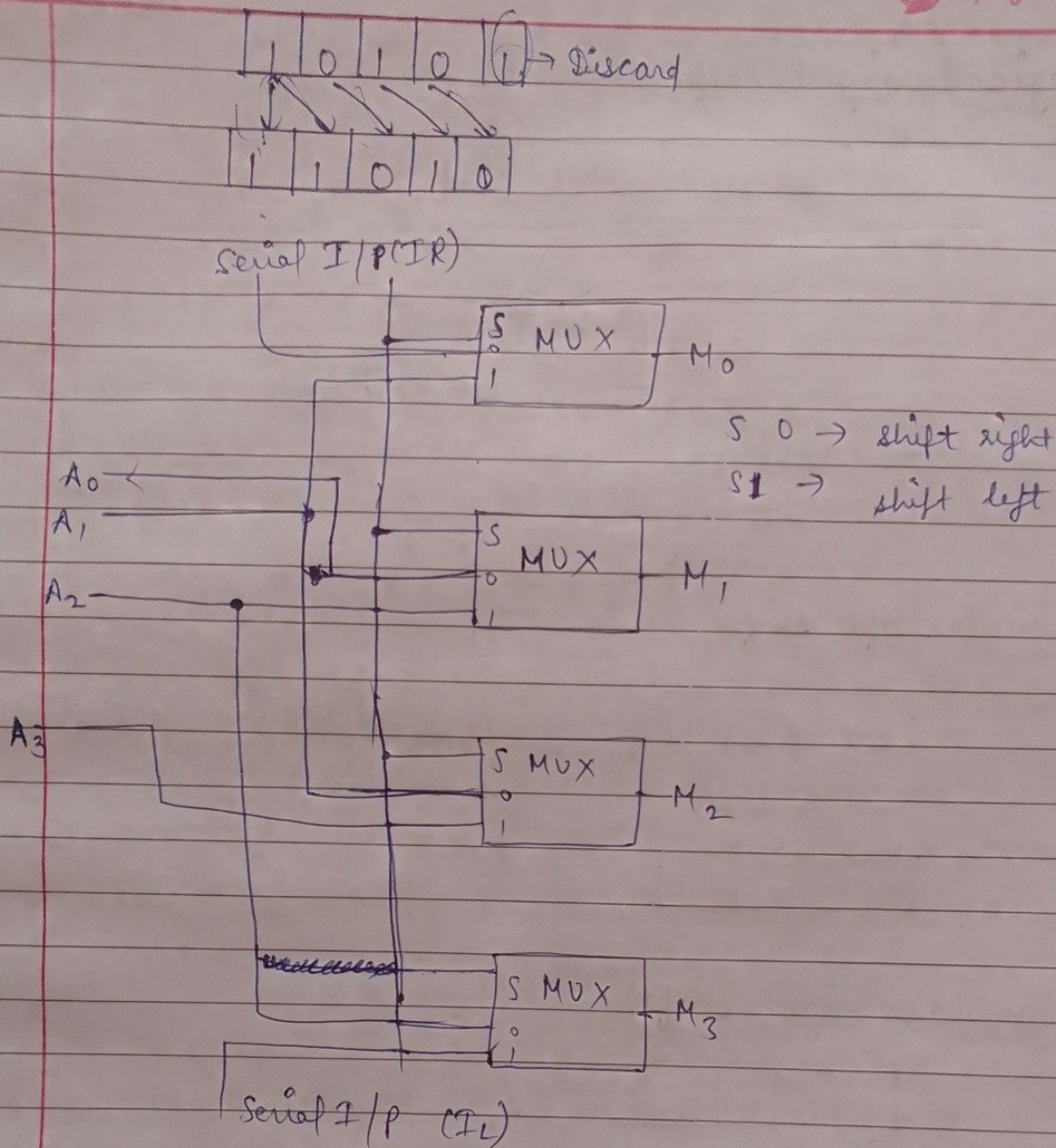
6.) 2's complement $\rightarrow R_2 \leftarrow \bar{R}_2 + 1$

[Register Transfer language]

~~Then~~

Three State Buffer :- 1) Zero state
 2) one state
 3) High Impedance state (open circuit)





Arithmetic operations

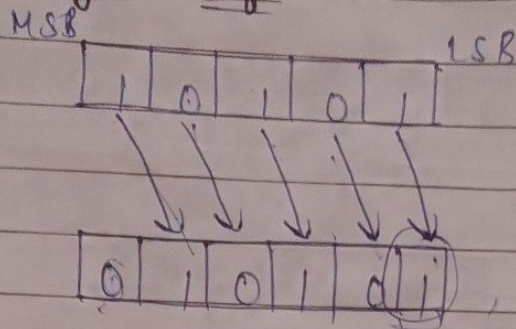
1) Addition:- Content of R₂ and R₁ is add and result store in R₃.

$$\begin{aligned} R_3 &\leftarrow R_1 + R_2 \\ R_3 &\leftarrow 10 + 5 \\ R_3 &\leftarrow 15 \end{aligned}$$

$$\begin{aligned} R_1 &= 10 \\ R_2 &= 5 \\ R_3 &= ? \end{aligned}$$

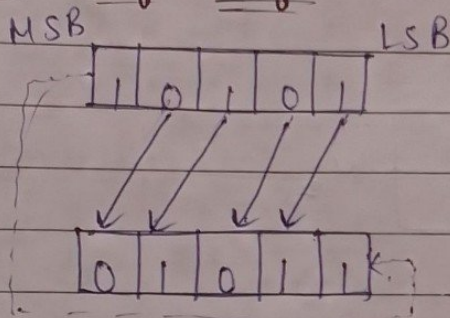
2) Subtraction:- Contents of R₂ subtracted from content of R₁ and result store in R₃.

2) logical right shift



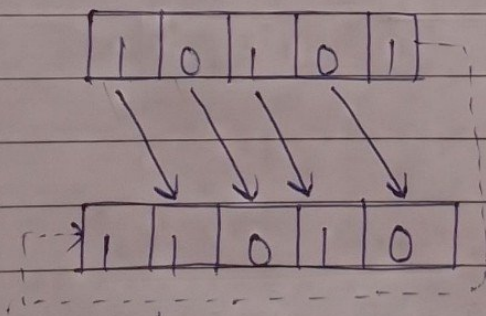
Position of each bit is move to right and left empty bit fill with 0 and right bit is discard. (MSB)
(LSB)

3) Circular left shift



Replace LSB with MSB (firstly shift to left)

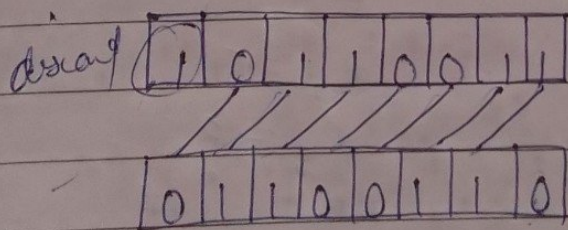
4) Circular right shift



first shift to right then left own bit is fill with empty bit.

5) Arithmetic left shift

(use for -ve numbers)



6) Arithmetic right shift

multiply
↑



Date _____

Page _____

Mask Operations (AND operations)

$$\begin{array}{r} 1010 \\ 1100 \\ \hline 1000 \end{array}$$

Insert Operation (OR operation) ↓ Addition

$$\begin{array}{r} 1010 \\ 1100 \\ \hline 0110 \end{array}$$

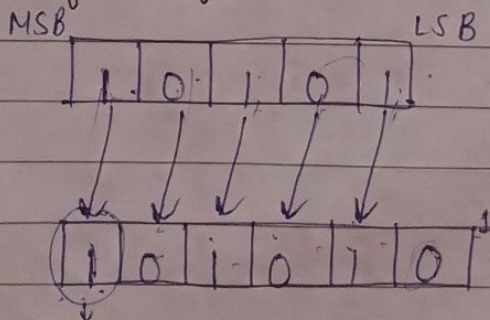
Clear operation (XOR operation)

$$\begin{array}{r} 1010 \\ 1100 \\ \hline 0110 \end{array}$$

$$\begin{array}{r} 1010 \\ 1010 \\ \hline 0000 \end{array}$$

28/8/25

logical left shift



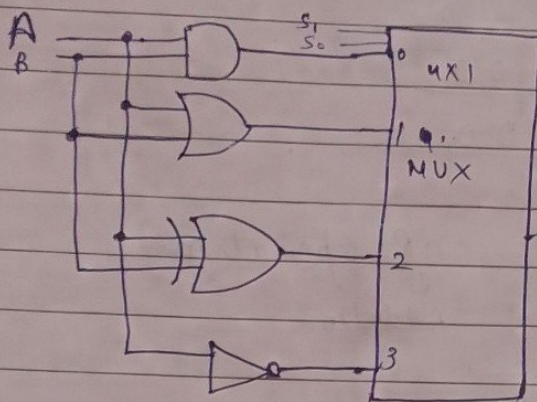
This bit is discard

Position of each bit is move to left and in bit (LSB) are fill with 0 and first bit (MSB) is discard.

Hardware Implementation

Demultiplexer = $1 \times 4 \rightarrow$ output

MUX - Multiplexer - $1 \times 1 \rightarrow$ output



S_1	S_0	o/p	operation
0	0	$E = A \cap B$	AND
0	1	$E = A \cup B$	OR
1	0	$E = A \oplus B$	XOR
1	1	$E = \bar{A}$	Complement

Selective Set

$A \rightarrow 1010 \leftarrow A \rightarrow \text{Register}$
 $B \rightarrow 1100 \leftarrow \text{operand (operation performed)}$
1110 after

if the B di value 1 $\rightarrow$ 1

if the B di value 0 $\rightarrow$ value of A

Selective Complement

if the B di value 0 $\rightarrow$ value of A (same)

if the B di value 1 $\rightarrow$ Complement of value of A

1010
 $\oplus 1100$

 0110

Selective clear

if the B di value 1 $\rightarrow$ 0

if the B di value 0 $\rightarrow$ same value of A

1010
 1100

 0010

No contents
↓

$$f \leftarrow 0$$

$$F \leftarrow A \wedge B$$

$$F \leftarrow A \wedge \bar{B}$$

$$f \leftarrow A$$

$$f \in \bar{A} \wedge B$$

$f \leftarrow B$

$$f \leftarrow A \oplus B$$

$$F \leftarrow A \vee B$$

$$f \leftarrow \overline{A \vee B}$$

$$f \leftarrow \overline{A \oplus B}$$

$$F \leftarrow \overline{B}$$

$$F \leftarrow A \vee \bar{B}$$

$f \leftarrow \bar{A}$

$$F \leftarrow \bar{A} \vee B$$

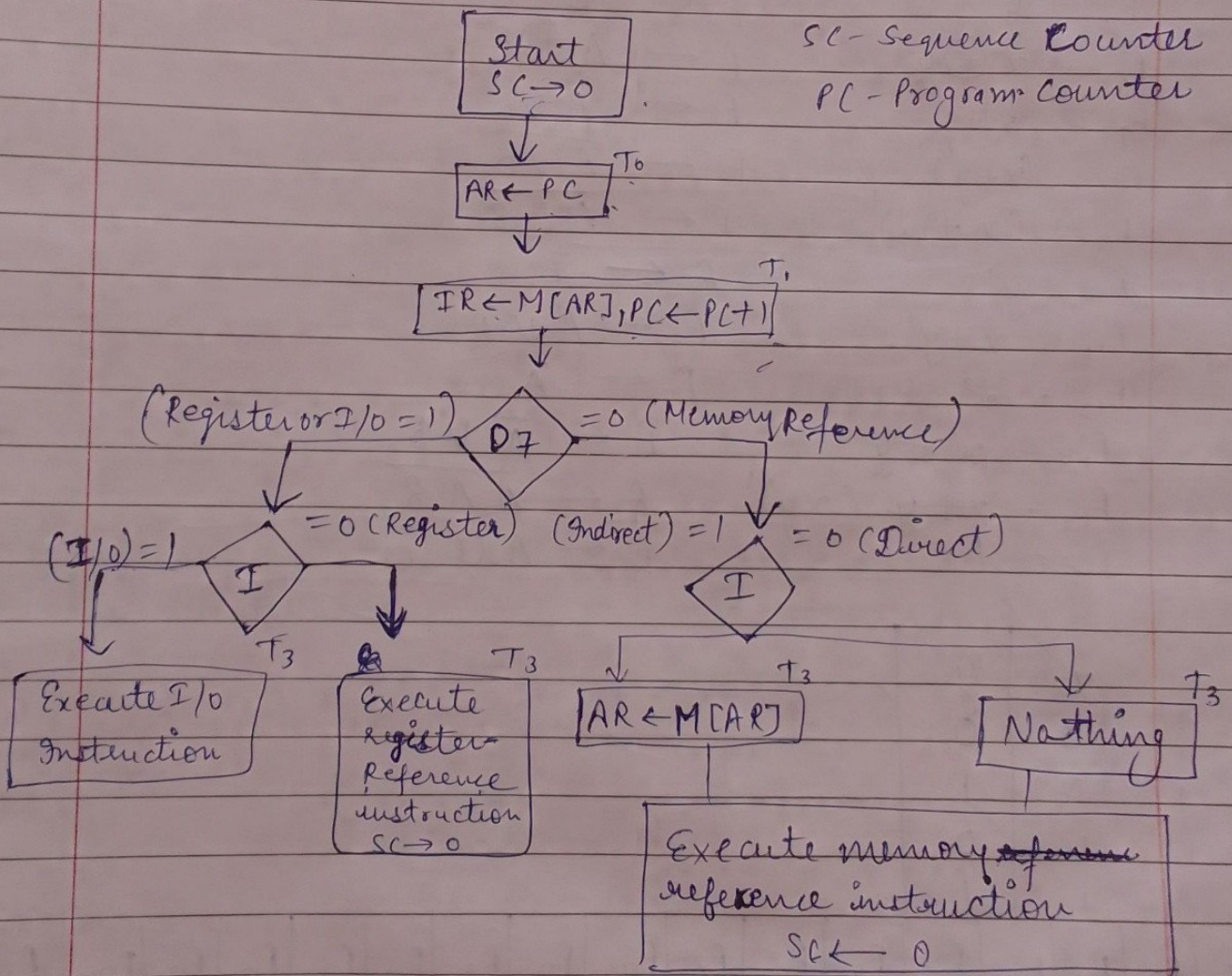
$$F \leftarrow A \wedge B$$

$f \leftarrow \text{All is}$

x	y	F_0	F_1	F_2	F_3	F_4	F_5	F_6	F_7	F_8	F_9	F_{10}	F_{11}	F_{12}	F_{13}	F_{14}	F_{15}
0	0	0	0	0	0	0	0	0	0	1	1	1	1	1	1	1	1
0	1	0	0	0	0	1	1	1	1	0	0	0	0	1	1	1	1
1	0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	1	1
1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	0	0	1

- 1.) Fetch the instruction from the memory.
- 2.) Decode the instruction
- 3.) Read the effective address from memory, If instruction has an indirect address
- 4.) Execute the instruction.

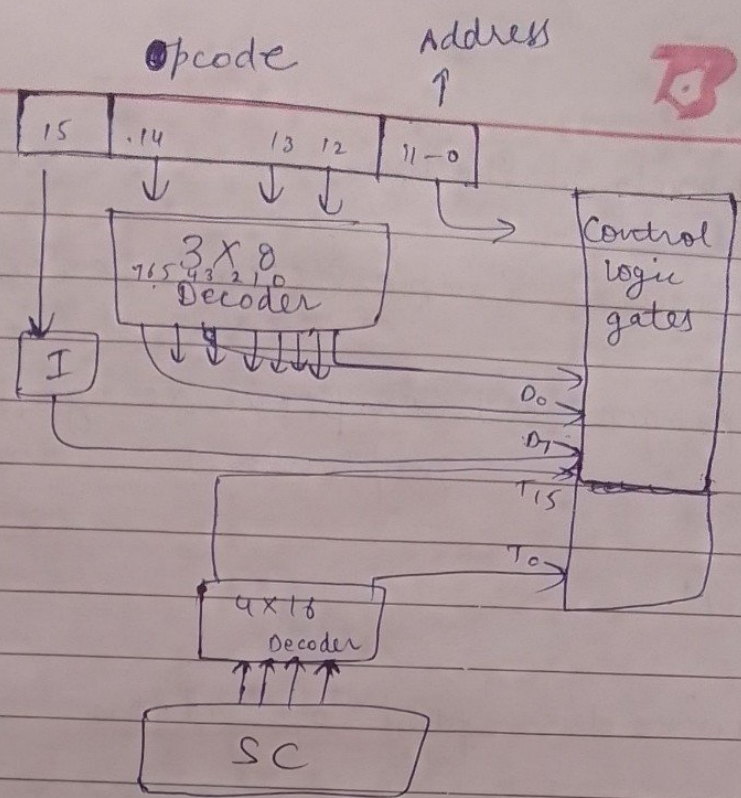
Flowchart



Microoperations :- four level operations made

logical operations	16 logic operations
$\wedge$ - And $\vee$ - OR $\oplus$ - XOR	$-$ → complement

Diagram
Explanation



3 X 8 decoder $\xrightarrow{\text{means}}$ 3 Input, $2^3 \rightarrow 8$
 $\rightarrow$ 8 output produce. $\xrightarrow{\text{handa ya}}$ control logic gates

4 X 16 decoder $\xrightarrow{\text{means}}$ 4 inputs kanda
 16 output produce. $\xrightarrow{\text{handa ya}}$ control logic gates

Instruction Cycle

when Instruction execute then which steps are taken

Instruction $\xrightarrow{\text{store}}$ memory

- 1.) Instruction $\rightarrow$ fetch ^(from) into memory
- 2.) $\rightarrow$ Decode
- 3.) $\rightarrow$ effective address
- 4.) $\rightarrow$ Execute

4. Complexity of Design:-

Hardwired:- Simple design, easy to implement.

Microprogrammend:- Complex design, difficult to implement.

5. Speed:-

Hardwired:- fast operation

Microprogrammend:- Slow operation, due to microcode decoding.

6. Debugging and Testing:-

Hardwired:- Difficult to Debug and Test

Microprogrammend:- Easy to ^{Debug} ~~Debug~~ and Test

7. Size and Cost:-

Hardwired:- Smaller size, low cost.

Microprogrammend:- larger size, High cost.

8. Maintenance and upgradability

Hardwired:- Difficult to upgrade and maintain.

Microprogrammend:- Easy to upgrade and maintain.

3) Program control instruction together with instructions that check status conditions.

4) Input and output instructions.

Timing & Control

master clock generator

1) Hardwired Control → fast operation

2) Microprogrammend control

Q. ^{ans} What is difference b/w Hardwired & microprogrammend?

Ans:-

1) Implementation:-

Hardwired:- fixed set of logic gates and circuits

Microprogrammend:- Micro code stored in the memory

2) Flexibility:-

Hardwired:- less flexible and difficult to modify.

Microprogrammend:- more flexible and easy to modify.

3) Instruction Set:-

Hardwired:- Supports limited Instruction set

Microprogrammend:- Supports complex Instruction set

SNA:- Skip the next instruction if AC is negative.

SZA:- skip the next instruction if AC is zero.

SZE:- skip the next instruction if E is zero.

HLT:- ^(stop) Halt Computer.

3.) Input / output:-

INP:- Input character to AC.

OUT:- Output character from AC.

SKI:- Skip ~~all~~ on input flag.

SKO:- Skip on output flag.

ION:- Interrupt on.

IOF:- Interrupt off.

Instruction Set completion:-

* A set of instructions are said to be complete if computer includes a sufficient number of instructions of following categories:-

- 1.) Arithmetic, logical and shift instructions.
- 2.) Instructions for moving information to and from memory and processor registers.

Examples:-



Date _____

Page _____

AND memory word to AC (accumulator)

ADD memory word to AC

LDA - Load memory word to AC

STA - Store the content of AC in memory

BUN - Branch unconditionally

BSA - Branch and save return address

ISZ - Increment and skip if zero.

2. Register Reference

CLA :- Clear AC.

CDE :- Clear E.

CMA :- Complement AC.

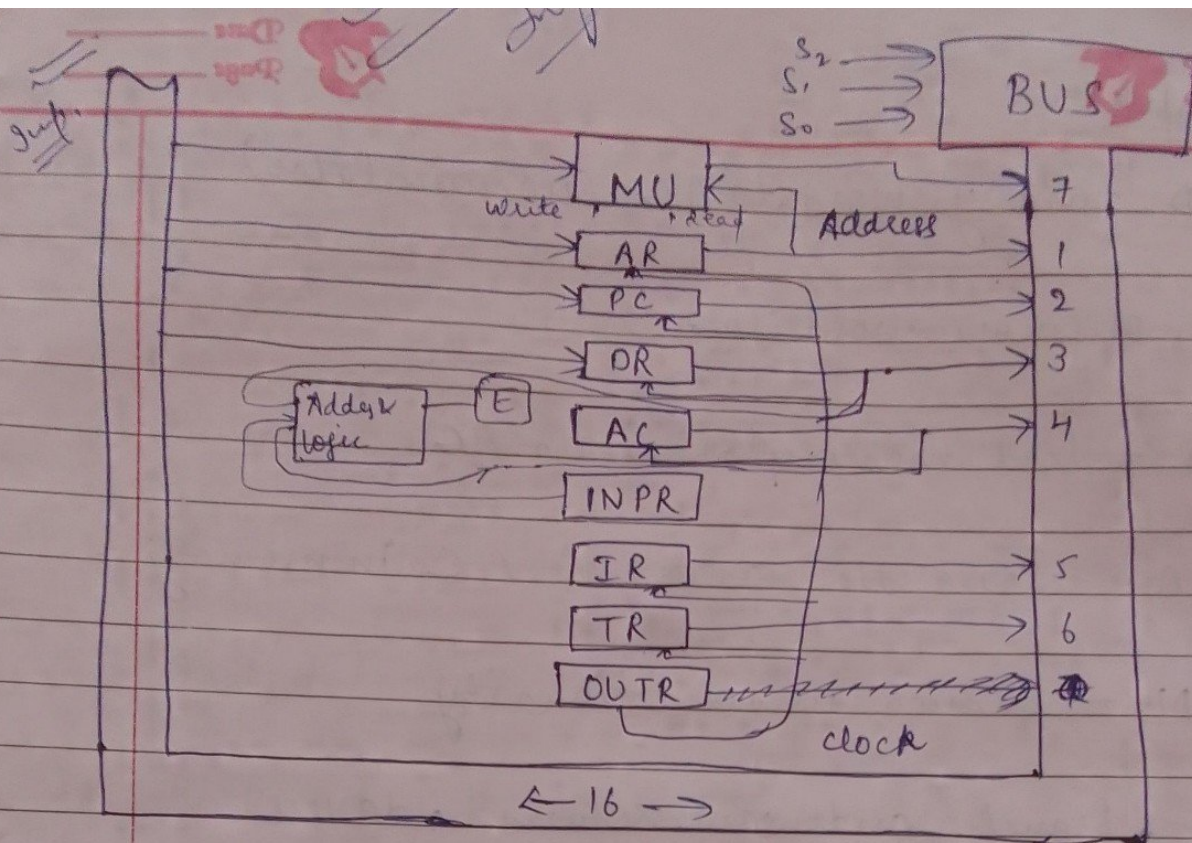
CME :- Complement E.

CIR :- Circular right AC & E.

CIL :- Circulate left AC & E.

INC :- Increment AC.

SPA :- Skip next instruction if AC is positive

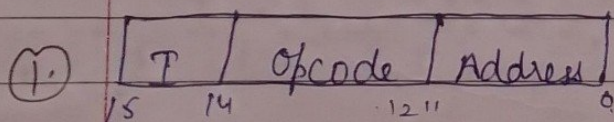


Computer Instructions

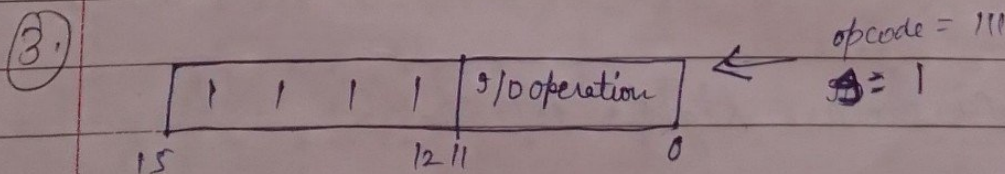
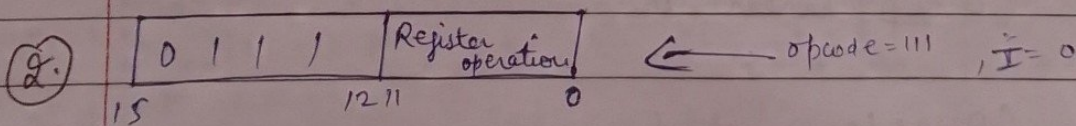
- 1.) Memory Reference - 16 bit
- 2.) Register Reference
- 3.) I/O

$I=0$ - Direct

$I=1$ - Indirect



0 0 0	Memory Reference Instructions
0 0 1	
0 1 0	
0 1 1	
1 0 0	
1 1 0	



- 1.) Memory Reference

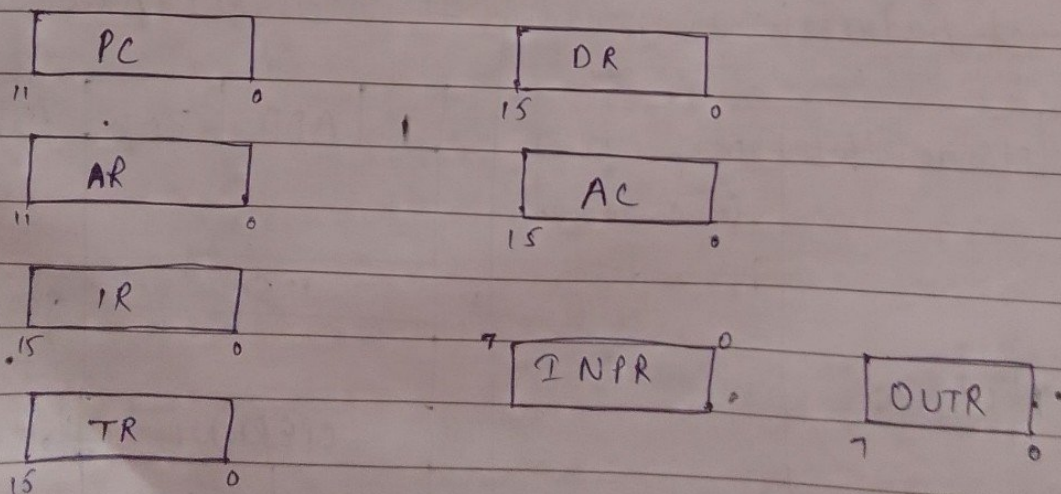
AND

$0 \times 0 = 0$
 $0 \times 1 = 0$
 $1 \times 0 = 0$
 $1 \times 1 = 1$

Computer Registers :-

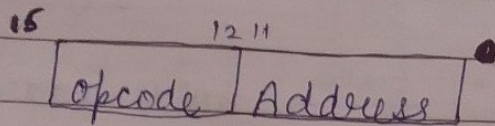
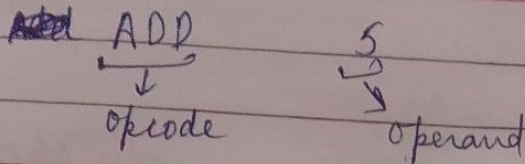
- 1) DR $\rightarrow$ Data Register - 16 bit ^{used to} holds memory operand
- 2) AR $\rightarrow$ Address Register - 12 bit ^{function} holds address for memory
- 3) AC $\rightarrow$ Accumulator - 16 bit
(Processor Register)
- 4) PC $\rightarrow$ Program counter - 12 bits $\rightarrow$ holds the address of instruction
- 5) TR $\rightarrow$ Temporary Register - 16 bits $\rightarrow$ holds temporary data
- 6) INPR $\rightarrow$ Input Register - 8 bit $\rightarrow$ holds input character
- 7) OUTF $\rightarrow$ Output Register - 8 bits $\rightarrow$ holds output character
- 8) IR $\rightarrow$ 16 bits $\rightarrow$ holds the instruction code

Instruction Register



4096 words 16 bits

eg:



Instruction format

$2^{12} \rightarrow 4096$ memory

$2^4 \rightarrow 16$

$12 - 15 \rightarrow 2^4$ bits for opcode

16 bits memory for opcode

$0 - 10 \rightarrow 12$ bits used for address

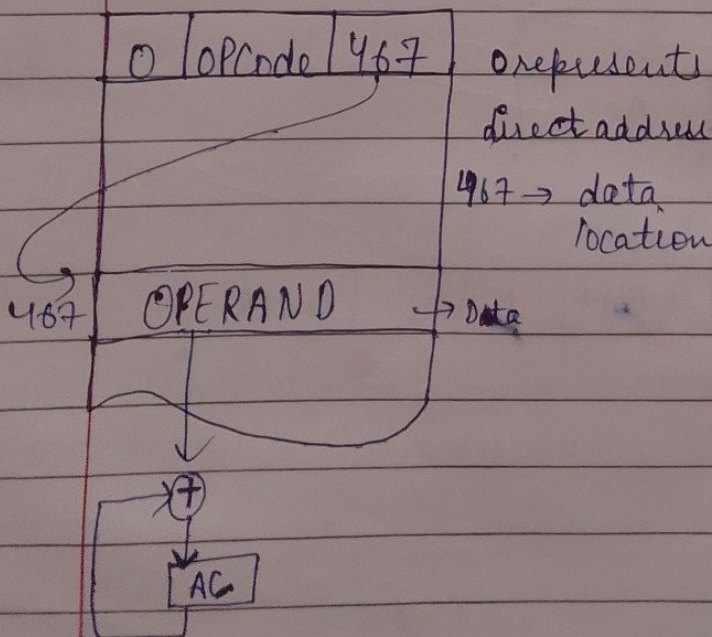
* Immediate Address

ADD 15

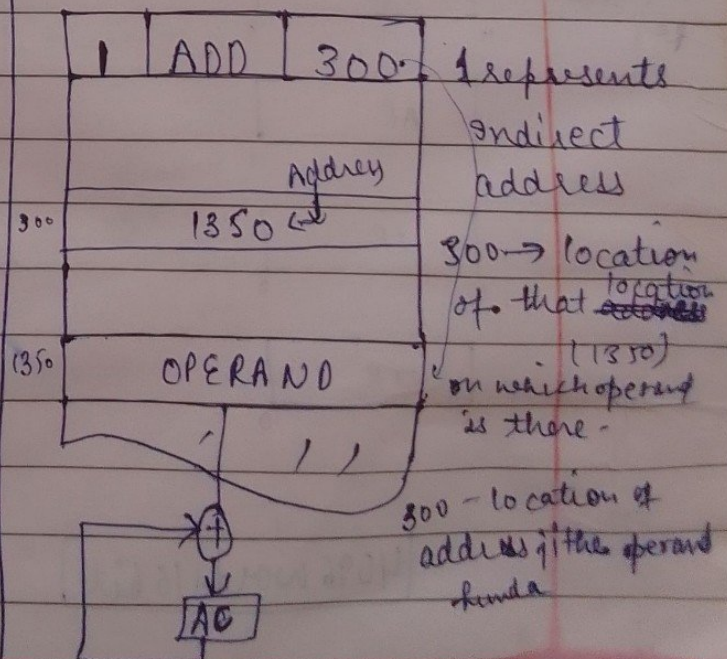
(Instruction deti naal data diinda)
(data naal pyo hunda opcode de)

Operation per formed on data and their address is immediate there at that time.

Direct Address: Data de address
Pyo hoaya jitha dianda data store aa.

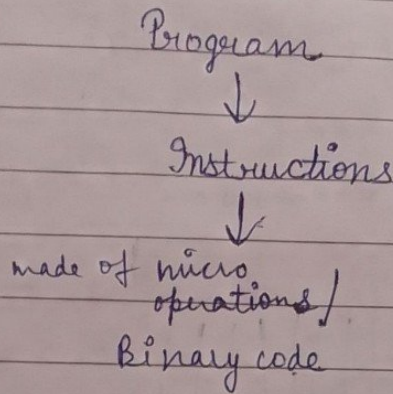


Indirect Address



Unit 3: computer Organization and design

* Instruction Codes



Instruction set :- group of ~~set~~ instructions

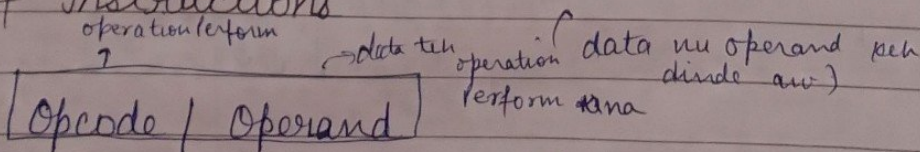
Data $\xrightarrow{\text{stores}}$ Memory
Instruction $\xrightarrow{\text{stores}}$ Memory

ALU $\rightarrow$ Arithmetic logic operations
actions $\xrightarrow{\text{perform}}$ addition, subtraction, multiplication

AC $\rightarrow$ Accumulator $\rightarrow$ (Temporary storage)

* Instruction code $\rightarrow$ set of bits, give instruction to computer to perform specific task.

* Parts of Instructions



only operation performed like addition, subtraction, division etc is opcode.

operation performed on data i.e. data is called operand.

$$\underline{x^4 + x^3 + 1}$$

$$1x^4 + 1x^3 + 0x^2 + 0x^1 + 1x^0$$

$$\underline{1 \quad 1 \quad 0 \quad 0 \quad 1}$$

$$11001$$

← minus one no. of bits

Polynomial X - no. of bits
add 1

X-OR

↓
Exclusive gate

$$\begin{array}{r}
 \underline{11001} \quad 10101010100000 \\
 11001 \downarrow \\
 011000 \\
 11001 \downarrow \\
 000011010 \\
 11001 \downarrow \\
 000110000 \\
 11001 \downarrow \\
 000010 \\
 \cancel{11001} \downarrow \\
 \cancel{11001}
 \end{array}$$

$$\begin{array}{r}
 \underline{11001} \quad 10101010100010 \\
 11001 \downarrow \\
 011000 \\
 11001 \downarrow \\
 000011010 \\
 11001 \downarrow \\
 00011001 \\
 11001 \downarrow \\
 000000
 \end{array}$$

$$m \rightarrow 10$$

$$r = 4$$

$$m + r = 10 + 4 = 14$$

2. 2-bit error :- When 2 bits are changed in data is called 2-bit error.

3. Burst error :- When data is changed in the sequence of more than 2 bits is called Burst error.

Detection methods :-

1. Parity Check :- It is of two types

Even Parity

Odd Parity

1) Even Parity :- like for eg. 1011 no. of one is ~~odd~~ there. there is odd no. & send data. one extra add & send data.

2) Odd Parity :- eg. 11001. no. one is odd.

2. Checksum :- we have given two data

Data word 1 $\rightarrow$ 1010 1001

Data word 2 $\rightarrow$ 0011 1001

First Add both data :- 1010 0010

One's complement $\rightarrow$ 1110 0010

One's complement $\rightarrow$ 0001

+

1101 $\rightarrow$ It is Checksum.

1111 1111

When comes one in data then there is no error.

3. Cycling Redundancy Check :- (CRC)

1010101010

(Data send karna chda vich Redundant bit add krke)

Error detection and Error Correction

Sender

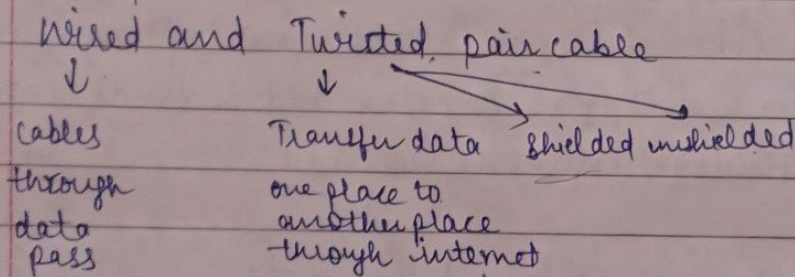
Receiver

1 0 1 0 1 0 1 0

1 0 1 0 1 1 1 0

- * Data Communication:- Sending or Receiving data, exchange data between two components.
- * Sender:- Sends the data
- * Receiver:- Receiving the data

Data paths:-



Wireless:- Radiowaves

Bluetooth

Microwaves

Infrared radi-

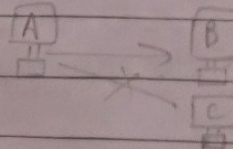
Fiber optics is fast data transfer.
(in the form of light)

- * Protocol:- set of rules which help data communication.

1) What is communicated? } Protocol defines these three ways.

2) How ?

3) When ?



Characteristics of data communication:-

- 1) Accuracy
- 2) Timeliness :- Data transfer at exact time which is given by us.
- 3) Zitter

Errors :- 1) One bit error (single bit error):- when in the data bit data is modified. (one bit changed)

- Floating point Representation:- There is change in position of decimal point
- * A real numbers in a computer can be represented in two ways (given above)

- 1) Fixed Point Representation:- In this method, decimal point is placed in a fixed place, a number ending with decimal is called whole number and numbers starting with decimal point is called fractional number.

For eg:- $45.00 \rightarrow$ whole number
 $\uparrow$ decimal extreme right

$00.45 \rightarrow$ fractional number
 $\downarrow$ decimal extreme left

- 2) Floating point Representation:- In this method, the position of decimal point is not specified by this a number can be expressed in many ways. $\rightarrow$ 3 parts

For eg:- $4.35 \downarrow$

$$435 \times 10^0$$

$$435 \times 10^1 \quad 43.5 \times 10^1$$

$$4.35 \times 10^2$$

$$.435 \times 10^3$$

1) Mantissa

2) Base

3) exponent

(3) $(10001)_2$
 $\downarrow$
 $(01110)_2$

$$\begin{array}{r} 01110 \\ + \quad 1 \\ \hline 01111 \end{array} \rightarrow (01111)_2$$

* 9's complement

(1) (873)

$$\begin{array}{r} 999 \\ - 873 \\ \hline 126 \end{array}$$

(2) (33815)

$$\begin{array}{r} 99999 \\ - 33815 \\ \hline 66184 \end{array}$$

10's complement

(1) 34968

$$\begin{array}{r} 99999 \\ - 34968 \\ \hline 65031 \end{array}$$

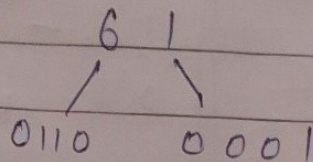
$$\begin{array}{r} 65031 \\ + \quad 1 \\ \hline 65032 \end{array}$$

A - B

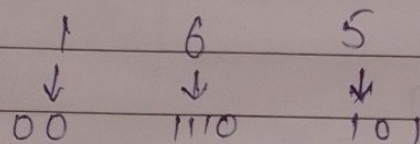
$$\begin{array}{r} A = 1011 \\ B = 1100 \\ \hline 1's - 0011 \\ 2's + \quad 1 \\ \hline 0100 \\ \hline 1011 \\ + 0100 \\ \hline 1111 \end{array}$$

• Fixed Point Representation $\rightarrow$ decimal point position fix

⑥ Hexadecimal to Binary
 $(61)_{16} \rightarrow (\quad)_2$



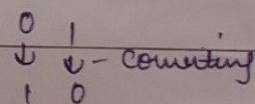
⑦ Octal to Binary
 $(165)_8 \rightarrow (\quad)_2$



Complement :- opposite

* 1's Complement

① $(1001)_2 \rightarrow (0110)_2$



② $(01001010)_2 \rightarrow (10110101)_2$

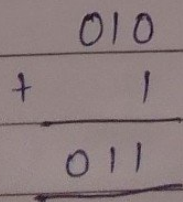
* 2's Complement

① $(101)_2$

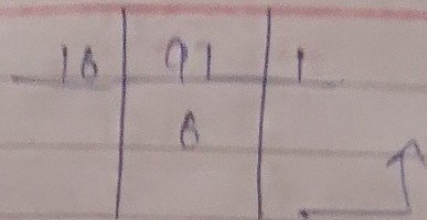


$(010)_2$

① First convert ^{into} 1's complement then add 1.



$\rightarrow (011)_2$



$(61)_{16}$

4) Octal to Decimal

$$(141)_8 \rightarrow (\quad)_{10}$$

$$1 \times 8^2 + 4 \times 8^1 + 1 \times 8^0 \Rightarrow 64 + 32 + 1 \Rightarrow 97$$

5) Hexadecimal to Decimal

$$(61)_{16} \rightarrow (\quad)_{10}$$

$$= 6 \times 16^1 + 1 \times 16^0$$

$$= 96 + 1$$

$$= 97$$

$(97)_{10}$

0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111
8	1000
9	1001
A	1010
B	1011
C	1100
D	1101
E	1110
F	1111